

4671340 Nan Ding
TELCOM 2310 Spring 2025
Lab 02: RDT

2.1 Getting Started

1. Record the IP addresses of your client and server nodes in Lab2.docx

Client node IP address: 10.132.7.2/24

```
Can't open user config file /home/fabric/work/fabric_config/ssh_co
ubuntu@client:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc fq_c
    link/ether fa:16:3e:95:52:ef brd ff:ff:ff:ff:ff:ff
    inet 10.30.6.160/23 metric 100 brd 10.30.7.255 scope global dyn
        valid_lft 84235sec preferred_lft 84235sec
    inet6 2001:1948:417:7:f816:3eff:fe95:52ef/64 scope global dyna
        valid_lft 86341sec preferred_lft 14341sec
    inet6 fe80::f816:3eff:fe95:52ef/64 scope link
        valid_lft forever preferred_lft forever
3: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq sta
    link/ether 3a:45:12:90:40:bd brd ff:ff:ff:ff:ff:ff
    inet 10.132.7.2/24 scope global enp7s0
        valid_lft forever preferred_lft forever
    inet6 fe80::3845:12ff:fe90:40bd/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@client:~$ █
```

Server node IP address: 10.138.136.2/24

```
ubuntu@server:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UP
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc fq
    link/ether fa:16:3e:62:41:56 brd ff:ff:ff:ff:ff:ff
    inet 10.30.6.14/23 metric 100 brd 10.30.7.255 scope global
        valid_lft 84224sec preferred_lft 84224sec
    inet6 2001:400:a100:3050:f816:3eff:fe62:4156/64 scope global
        valid_lft 86369sec preferred_lft 14369sec
    inet6 fe80::f816:3eff:fe62:4156/64 scope link
        valid_lft forever preferred_lft forever
3: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mc
    link/ether 02:07:4e:13:d5:7e brd ff:ff:ff:ff:ff:ff
    inet 10.138.136.2/24 scope global enp7s0
        valid_lft forever preferred_lft forever
    inet6 fe80::7:4eff:fe13:d57e/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@server:~$
```

Finding RTT

2. Enter the average RTT between your nodes in Lab2.docx

On Client Node, the average RTT is 65.882 ms

```
-----
ubuntu@client:~$ ping 2001:400:a100:3050:f816:3eff:fe6
PING 2001:400:a100:3050:f816:3eff:fe62:4156(2001:400:a
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
64 bytes from 2001:400:a100:3050:f816:3eff:fe62:4156:
^C
--- 2001:400:a100:3050:f816:3eff:fe62:4156 ping statis
11 packets transmitted, 11 received, 0% packet loss, t
rtt min/avg/max/mdev = 65.679/65.882/67.519/0.518 ms
ubuntu@client:~$
```

Go-Back-N File Transfer Programs

3. Copy and paste the output from your server and client into Lab2.docx

On Client Node:

```
ubuntu@server: ~  X  ubuntu@client: ~  X  RDTLab.ipynb

    valid_lft forever preferred_lft forever
ubuntu@client:~$ python3 GBN_Client.py -a 10.138.136.2 -f test_file_100KB.txt
Final packet with seq 101 acknowledged. Time to quit...

=== Final Stats ===
Bytes transferred: 100017
Time Elapsed: 5.2050254344940186 sec
Throughput: 0.1537237444984323 Mbps
ubuntu@client:~$
```

On Server Node:

```
ubuntu@server: ~  X  ubuntu@client: ~  X  RDTLab.ipynb  X  CreateSlice.ipynb

    valid_lft forever preferred_lft forever
ubuntu@server:~$ python3 GBN_Server.py -f copy_100KB.txt
Opened Server socket
Bound socket on port 12000
Packet 100      Bytes Recvd 100017      Time Elapsed 5.103011846542358  Throughput 0.1567968141289241 Mbps
Got last packet with seq 101. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...
Bytes transferred: 100017
Time Elapsed: 5.153885841369629 sec
Throughput: 0.1552490731512529 Mbps
ubuntu@server:~$
```

Verify the file was copied correctly on server and client side

```
ubuntu@server: ~  X  ubuntu@client: ~  X  RDTLab.ipynb  X  CreateSlice.ipynb

ubuntu@client:~$
ubuntu@client:~$ python3 GBN_Server.py -f returned_100KB.txt
Opened Server socket
Bound socket on port 12000
Packet 100      Bytes Recvd 100017      Time Elapsed 5.095552921295166  Throughput 0.15702633499420607 Mbps
Got last packet with seq 101. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...
Bytes transferred: 100017
Time Elapsed: 5.1464080810546875 sec
Throughput: 0.1554746509406271 Mbps
ubuntu@client:~$
ubuntu@client:~$
ubuntu@client:~$ diff test_file_100KB.txt returned_100KB.txt
ubuntu@client:~$
```

```
ubuntu@server: ~  X  ubuntu@client: ~  X  RDTLab.ipynb

ubuntu@server:~$
ubuntu@server:~$ python3 GBN_Client.py -a 10.132.7.2 -f copy_100KB.txt
Final packet with seq 101 acknowledged. Time to quit...

=== Final Stats ===
Bytes transferred: 100017
Time Elapsed: 5.197406530380249 sec
Throughput: 0.15394908890097173 Mbps
ubuntu@server:~$
ubuntu@server:~$
ubuntu@server:~$ diff test_file_100KB.txt copy_100KB.txt
diff: returned 100KB.txt: No such file or directory
ubuntu@server:~$ diff test_file_100KB.txt copy_100KB.txt
ubuntu@server:~$
```

Window Size and Throughput

4. Enter your expected throughput for a window size of 1 packet in Lab2.docx

Throughput = (window_size * packet_size) / RTT = (1 packet * 8000 bits) / 0.051534905 sec/packet = 155234.593 bits/sec = 0.155234593 Mbps $\approx$ 0.155 Mbps

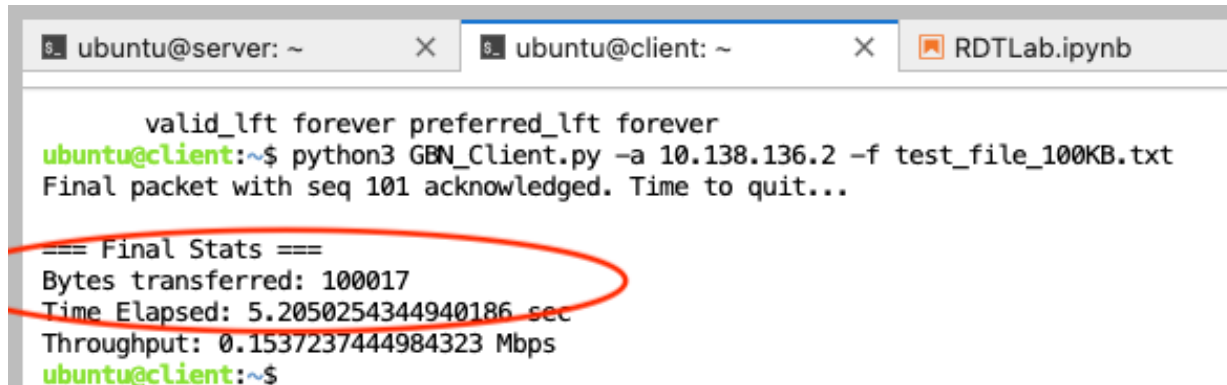
Where,

window_size = 1 packet ,

packet_size = 1000 bytes = 1000 bytes * 8 bits/bytes = 8000 bits,

RTT = Time Elapsed / number of packets = 5.2050254 sec / 101 packets = 0.051534905 sec/packet

(From the lab2, we can see, Time Elapsed is about 5.2050254 seconds, number of packets is about 101, where: $(100017 \text{ bytes} / 1000 \text{ bytes/packet}) = 100.017 \approx 101 \text{ packets}$)



```
valid_lft forever preferred_lft forever
ubuntu@client:~$ python3 GBN_Client.py -a 10.138.136.2 -f test_file_100KB.txt
Final packet with seq 101 acknowledged. Time to quit...
=== Final Stats ===
Bytes transferred: 100017
Time Elapsed: 5.2050254344940186 sec
Throughput: 0.1537237444984323 Mbps
ubuntu@client:~$
```

5. Does your measured result from 3a above match the expected result that you calculated? ****Comment in Lab2.docx****

Answer:

Not same, but close. The measured result is about 0.153 Mbps, the expected result is 0.155 Mbps.

6. Calculate the window size needed to achieve a rate of 1 Mbps based on a packet size of 1000 bytes and the RTT between your nodes. ****Enter your answer in Lab2.docx****

If we want to get a throughput of 1 Mbps, the window size should be 7 packets.

Because, $\text{window_size} = (\text{Throughput} * \text{RTT}) / \text{packet_size} = (1 * 10^6 \text{ bits/sec} * 0.051534905 \text{ sec/packet}) / 8000 \text{ bits} = 6.44186312 \text{ packets} \approx 7 \text{ packets}$ (window size should be an integer)

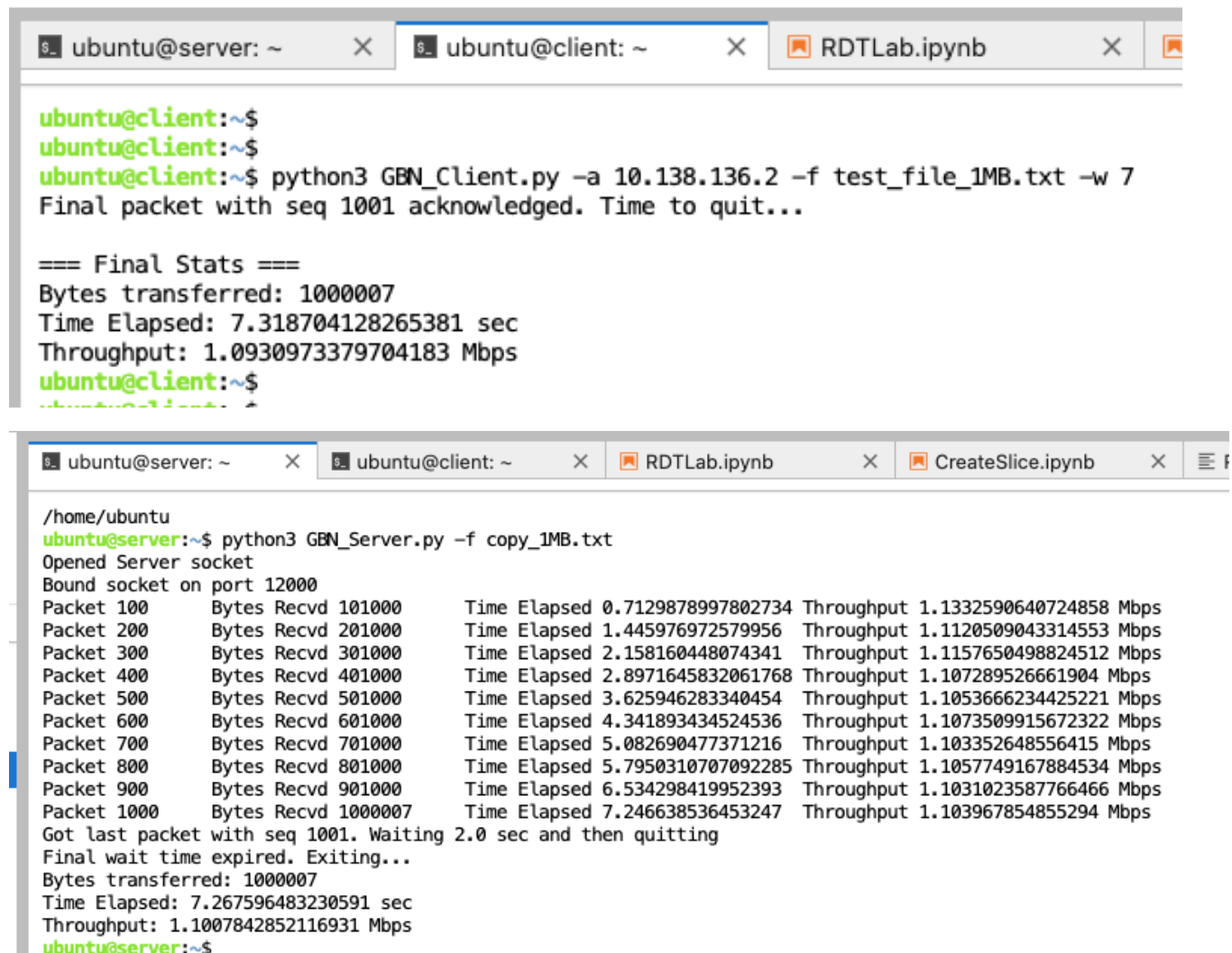
Where:

Expected Throughput = 1 Mbps = $1 * 10^6 \text{ bits/sec}$

RTT = 0.051534905 sec/packet

packet_size= 1000 bytes = 8000 bits

7. **Copy and paste your output in Lab2.docx Do the results match your expectation?**



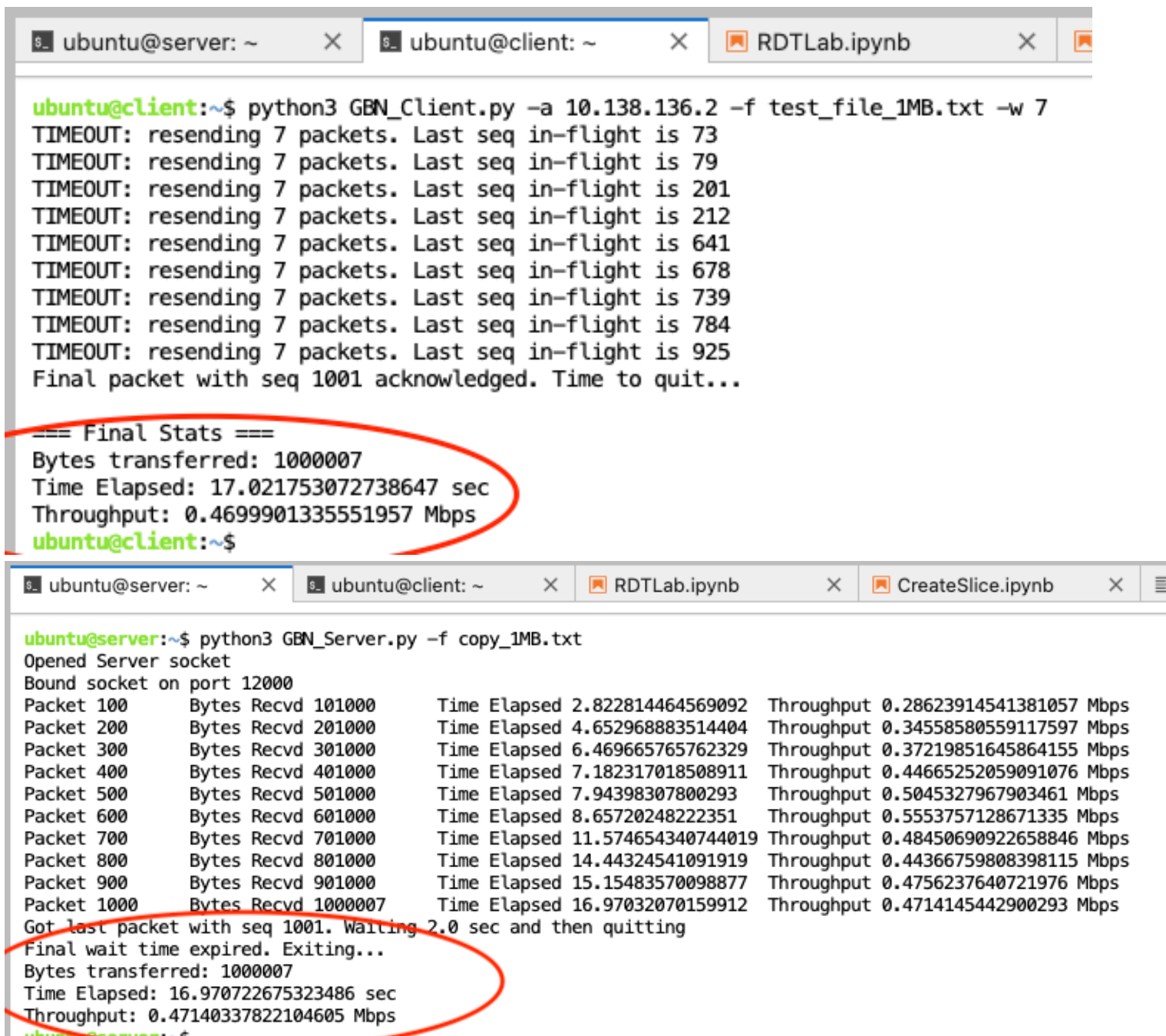
```
ubuntu@client:~$  
ubuntu@client:~$  
ubuntu@client:~$ python3 GBN_Client.py -a 10.138.136.2 -f test_file_1MB.txt -w 7  
Final packet with seq 1001 acknowledged. Time to quit...  
  
=== Final Stats ===  
Bytes transferred: 1000007  
Time Elapsed: 7.318704128265381 sec  
Throughput: 1.0930973379704183 Mbps  
ubuntu@client:~$
```

```
/home/ubuntu  
ubuntu@server:~$ python3 GBN_Server.py -f copy_1MB.txt  
Opened Server socket  
Bound socket on port 12000  
Packet 100      Bytes Recvd 101000      Time Elapsed 0.7129878997802734      Throughput 1.1332590640724858 Mbps  
Packet 200      Bytes Recvd 201000      Time Elapsed 1.445976972579956      Throughput 1.1120509043314553 Mbps  
Packet 300      Bytes Recvd 301000      Time Elapsed 2.158160448074341      Throughput 1.1157650498824512 Mbps  
Packet 400      Bytes Recvd 401000      Time Elapsed 2.8971645832061768      Throughput 1.107289526661904 Mbps  
Packet 500      Bytes Recvd 501000      Time Elapsed 3.625946283340454      Throughput 1.1053666234425221 Mbps  
Packet 600      Bytes Recvd 601000      Time Elapsed 4.341893434524536      Throughput 1.1073509915672322 Mbps  
Packet 700      Bytes Recvd 701000      Time Elapsed 5.082690477371216      Throughput 1.103352648556415 Mbps  
Packet 800      Bytes Recvd 801000      Time Elapsed 5.7950310707092285      Throughput 1.1057749167884534 Mbps  
Packet 900      Bytes Recvd 901000      Time Elapsed 6.534298419952393      Throughput 1.1031023587766466 Mbps  
Packet 1000     Bytes Recvd 1000007      Time Elapsed 7.246638536453247      Throughput 1.103967854855294 Mbps  
Got last packet with seq 1001. Waiting 2.0 sec and then quitting  
Final wait time expired. Exiting...  
Bytes transferred: 1000007  
Time Elapsed: 7.267596483230591 sec  
Throughput: 1.1007842852116931 Mbps  
ubuntu@server:~$
```

Yes, the result nearly matches the expectation, from the screenshot, we can see, the measured Throughput at client node is about 1.093 Mbps with the window size change to 7 packets.

Loss Emulation

8. ****Copy and paste the output from your server and client into Lab2.docx****



The screenshot displays two terminal windows. The top window, titled 'ubuntu@client: ~', shows the execution of a Python script 'GBN_Client.py' with arguments '-a 10.138.136.2 -f test_file_1MB.txt -w 7'. The output indicates multiple timeouts, with the last sequence number in-flight increasing from 73 to 925. A red circle highlights the 'Final Stats' section, which reports: 'Bytes transferred: 1000007', 'Time Elapsed: 17.021753072738647 sec', and 'Throughput: 0.4699901335551957 Mbps'. The bottom window, titled 'ubuntu@server: ~', shows the execution of 'GBN_Server.py -f copy_1MB.txt'. It displays a table of packet transfer statistics for packets 100 through 1000. A red circle highlights the final summary statistics: 'Bytes transferred: 1000007', 'Time Elapsed: 16.970722675323486 sec', and 'Throughput: 0.47140337822104605 Mbps'.

```
ubuntu@client:~$ python3 GBN_Client.py -a 10.138.136.2 -f test_file_1MB.txt -w 7
TIMEOUT: resending 7 packets. Last seq in-flight is 73
TIMEOUT: resending 7 packets. Last seq in-flight is 79
TIMEOUT: resending 7 packets. Last seq in-flight is 201
TIMEOUT: resending 7 packets. Last seq in-flight is 212
TIMEOUT: resending 7 packets. Last seq in-flight is 641
TIMEOUT: resending 7 packets. Last seq in-flight is 678
TIMEOUT: resending 7 packets. Last seq in-flight is 739
TIMEOUT: resending 7 packets. Last seq in-flight is 784
TIMEOUT: resending 7 packets. Last seq in-flight is 925
Final packet with seq 1001 acknowledged. Time to quit...

=== Final Stats ===
Bytes transferred: 1000007
Time Elapsed: 17.021753072738647 sec
Throughput: 0.4699901335551957 Mbps
ubuntu@client:~$

ubuntu@server:~$ python3 GBN_Server.py -f copy_1MB.txt
Opened Server socket
Bound socket on port 12000
Packet 100      Bytes Recvd 101000      Time Elapsed 2.822814464569092      Throughput 0.28623914541381057 Mbps
Packet 200      Bytes Recvd 201000      Time Elapsed 4.652968883514404      Throughput 0.34558580559117597 Mbps
Packet 300      Bytes Recvd 301000      Time Elapsed 6.469665765762329      Throughput 0.37219851645864155 Mbps
Packet 400      Bytes Recvd 401000      Time Elapsed 7.182317018508911      Throughput 0.44665252059091076 Mbps
Packet 500      Bytes Recvd 501000      Time Elapsed 7.94398307800293      Throughput 0.5045327967903461 Mbps
Packet 600      Bytes Recvd 601000      Time Elapsed 8.65720248222351      Throughput 0.5553757128671335 Mbps
Packet 700      Bytes Recvd 701000      Time Elapsed 11.574654340744019      Throughput 0.48450690922658846 Mbps
Packet 800      Bytes Recvd 801000      Time Elapsed 14.44324541091919      Throughput 0.44366759808398115 Mbps
Packet 900      Bytes Recvd 901000      Time Elapsed 15.15483570098877      Throughput 0.4756237640721976 Mbps
Packet 1000     Bytes Recvd 1000007     Time Elapsed 16.97032070159912      Throughput 0.4714145442900293 Mbps
Got last packet with seq 1001. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...
Bytes transferred: 1000007
Time Elapsed: 16.970722675323486 sec
Throughput: 0.47140337822104605 Mbps
```

9. How many timeouts do you observe? How many would you expect (based on loss rate and total number of 1000-byte packets needed to transfer a 1 MB file)? ****Answer in Lab2.docx****

Observe: according to screenshot from Client and Server Node, **there are 9 timeouts.**

Expect: **there could be about 10 timeouts.** Because:

Number of packets shall be transferred = File size / packet size = 1 MB / 1000 byte/packet = 1000 packets, since the loss rate is 1%, so the number of lost packets = 1% * 1000 packets = 10 packets, in Go-back-N protocol, each time when packet lost, there will be a timeout and the entire window will be retransmitted. So, there will be about 10 timeouts basically.

Termination

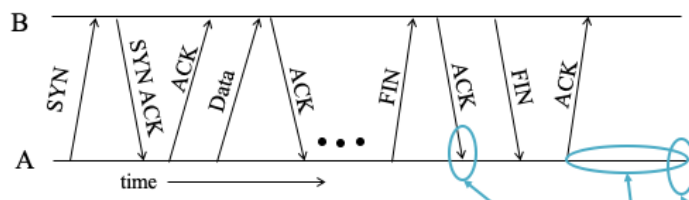
10. Why do you think this timeout is needed? ****Answer in Lab2.docx****

In GBN_Server.py, the code related to “waiting 2 seconds before existing” happens here:

Like taught by teacher in class, when server quit quickly,

if client did not receive the ACK, it will retransmit the packets and keep waiting ACK, also the retransmitted packets will not be received by server. Besides, when client finish its data transmission, it will send FIN to server, once server receive the FIN, it shall send ACK back to client, so server also need to wait in case the ACK is lost, server needs to retransmit the ACK. Until Client receive the ACK and FIN from server, it will send ACK back to server, so when server get ACK from client, connection is closed finally.

Closing a TCP Connection: Normal Termination Example



```
GBN_Server.py  X  ubuntu@client: ~  X  ubuntu@server: ~  X  RDTRLab.ipynb
54         bytes_recvd += len(data)
55
56         # Periodic reporting
57         if pkt_header.seq_num > 0 and pkt_header.seq_num % 100 == 0:
58             cur_time = time.time()
59             elapsed_time = cur_time - start_time
60             throughput = (bytes_recvd * 8) / (elapsed_time) / 1000000
61             print("Packet {} \t Bytes Recvd {} \t Time Elapsed {} \t Throughput " \
62                   "{} Mbps".format(pkt_header.seq_num, bytes_recvd, elapsed_time,
63                                   throughput))
64         elif pkt_header.type == PacketHeader.TYPE_END:
65             # Data transfer is done. Close file and set final timeout
66             print("Got last packet with seq {}. Waiting {} sec and then " \
67                   "quitting".format(pkt_header.seq_num, final_timeout))
68             outfile.close()
69             end_time = time.time()
70             server_socket.settimeout(final_timeout)
71
72         # Update expected sequence number and stats
73         expected_seq += 1
74
75         # Else, received packet is not the next one I'm expecting, so do
76         # nothing (i.e. drop the packet). Recall that we can improve
77         # performance by buffering instead (like TCP does) but this requires a
78         # bit more logic
```

11. If you run this scenario several times (re-run the server and client, without changing the loss settings), you will likely encounter the scenario where the sender (client) does not terminate but instead keeps trying to retransmit the last packet (in this case, you should `CTRL+C` the client to kill the program). What is the specific event that leads to this outcome? ****Answer in Lab2.docx****

With the high loss rate, not only the data but also the ACK will have high possibility to be lost. And the low timeout setting will also trigger retransmit more easily even the data/ACK is just delayed instead of lost. All these 2 parameters will lead to the high retransmit rate.

And in Go-back-N protocol, receiver will transmit new data when the ACK of last packet in the window is received. So if the last packet lost, sender will resend the packets of whole window size and cannot move forward. So that is the reason why we can see so many times about "last seq in flight is 10" when the loss rate is high and timeout is very low.

ubuntu@client: ~ X ubuntu@server: ~ X RDTLab.ipynb X GBN_S

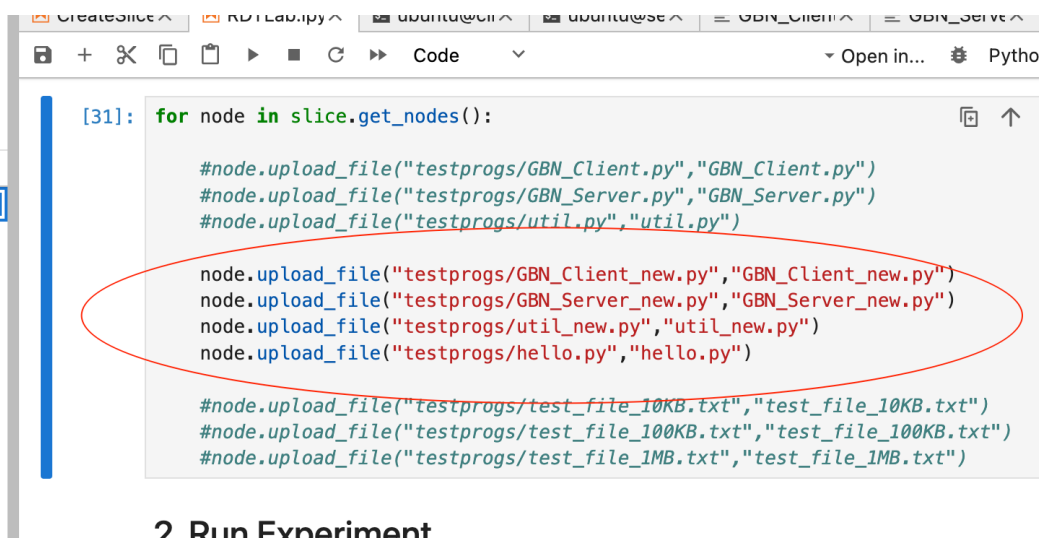
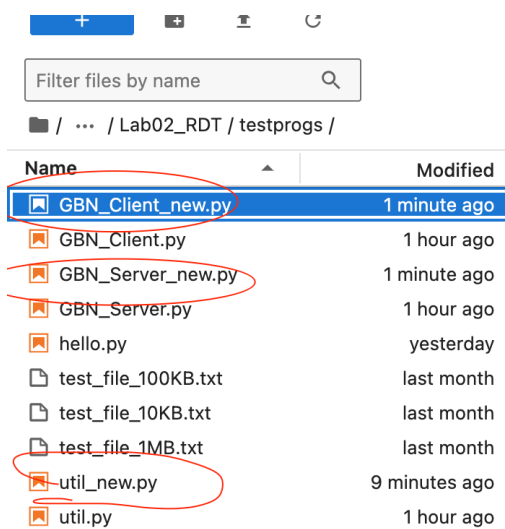
[illegible]

Bonus: Improve the programs

- A simple idea (5 points):
 - Instead of requiring the server specify a file name to save the file as, have the client send a special first packet that gives the file name (this will likely require adding a new packet type in util.py)

Below are the steps of this question solving:

a) in order not to effect the original python files of “GBN_Client”, “GBN_Server”, “util”, I created 3 modified python files “GBN_Client_new”, “GBN_Server_new”, “util_new” and uploaded them to each node.



b) Modified the “util_new.py”, add code of file type FILENAME

```

from scapy.all import Packet
from scapy.all import IntField

class PacketHeader(Packet):
    TYPE_DATA = 1
    TYPE_ACK = 2
    TYPE_END = 3
    TYPE_FILENAME = 4    # new add

    header_len = 12
    name = "PacketHeader"
    fields_desc = [
        IntField("type", 0),
        IntField("seq_num", 0),
        IntField("length", 0),
    ]

```

c) Modified the “GBN_Client.py”, add codes related to send packet of filename

```

38
39     # new add Send filename packet
40     filename_data = args.filename.encode()
41     filename_pkt_header = PacketHeader(type=PacketHeader.TYPE_FILENAME, seq_num=seq,
length=len(args.filename))
42     filename_pkt=filename_pkt_header / filename_data
43     client_socket.sendto(bytes(filename_pkt), (args.address, args.port))
44     print(f"Send:Filename_PacketHeader: type={PacketHeader.TYPE_FILENAME}, seq_num={seq}, length=
{len(args.filename)}, filename={args.filename}")
45     seq += 1
46

```

d) Modified the “GBN_Server.py”, add codes related to receive packet of filename

```

5
7     # new add Handle filename packet
3     if pkt_header.type == PacketHeader.TYPE_FILENAME:
9         filename = data.decode().strip()
0         print(f"Receive: PacketHeader: type={pkt_header.type}, seq_num=
{pkt_header.seq_num}, length={pkt_header.length}, filename={filename}")
1         if not filename:
2             print("Error: Filename is empty. Skipping file creation.")
3             continue # Skip opening the file if the filename is empty
4         try:
5             outfile = open(filename, 'wb')
6         except Exception as e:
7             print(f"Error opening file: {e}")
8             continue
9         ack_header = PacketHeader(seq_num=expected_seq, length=0)
0         server_socket.sendto(bytes(ack_header), addr)
1         print(f"Sent ACK for filename packet: seq={expected_seq}")
2         expected_seq += 1
3         continue # Skip the rest of the loop for filename packet
4

```

Here is the screenshot of the Client node:

The 0th sequence number of the packet is the filename packet with the length 19 bytes and "test_file_100KB.txt" name, and client also gets ACK of seq=0 from server successfully.

```

KeyboardInterrupt
ubuntu@client:~$ ^C
ubuntu@client:~$ python3 GBN_Client_new.py -a 10.131.2.2 -f test_file_100KB.txt
Send:Filename_PacketHeader: type=4, seq_num=0, length=19, filename=test_file_100KB.txt
Send:PacketHeader: type=1, seq_num=2, length=1000
Received ACK: seq=0
Filename packet acknowledged by server.
Received ACK: seq=2
Send:PacketHeader: type=1, seq_num=3, length=1000
Received ACK: seq=3
Send:PacketHeader: type=1, seq_num=4, length=1000
Received ACK: seq=4
Send:PacketHeader: type=1, seq_num=5, length=1000

```

Here is the screenshot of the Server node:

Server receive the seq=0, and filename = test_file_100KB.txt packet and also sent ACK back to client successfully.

```
CreateSlice X RDTLab.ipyn X ubuntu@cli X ubuntu@se X GBN_Client X GBN_Server X util_new.py X

ubuntu@server:~$ python3 GBN_Server_new.py
Opened Server socket
Bound socket on port 12000
Receive: PacketHeader: type=4, seq_num=0, length=19, filename=test_file_100KB.txt
Sent ACK for filename packet: seq=0
Packet 100 Bytes Recvd 100000 Time Elapsed 3.6160190105438232 Throughput 0.22123777493074787 Mbps
Got last packet with seq 102. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...
Bytes transferred: 100017
Time Elapsed: 3.68915057182312 sec
Throughput: 0.21688895164953526 Mbps
ubuntu@server:~$
```

- More involved (15 points):
 - Change the Go-Back-N implementation to buffer out-of-order packets instead of discarding them, so the sender does not need to re-send the entire window on loss (similar to TCP)

Below are the steps of this question solving:

a) in order not to effect the original python files of “GBN_Client”, “GBN_Server”, “util”, I created 2 modified python files “GBN_Client_buffer.py”, “GBN_Server_buffer.py”, and uploaded them to each node.

Filter files by name

/ ... / Lab02_RDT / testprogs /

Name	Modified
util.py	10 hours ago
util_new.py	8 hours ago
test_file_1MB.txt	last month
test_file_10KB.txt	last month
test_file_100KB.txt	last month
hello.py	2 hours ago
GBN_Server.py	10 hours ago
GBN_Server_new.py	8 hours ago
GBN_Server_buffer.py	2 minutes ago
GBN_Client.py	10 hours ago
GBN_Client_new.py	8 hours ago
GBN_Client_buffer.py	6 minutes ago

Code

```
#node.upload_file("testprogs/GBN_Client.py","GBN_Client.py")
#node.upload_file("testprogs/GBN_Server.py","GBN_Server.py")
#node.upload_file("testprogs/util.py","util.py")

#node.upload_file("testprogs/GBN_Client_new.py","GBN_Client_new.py")
#node.upload_file("testprogs/GBN_Server_new.py","GBN_Server_new.py")
#node.upload_file("testprogs/util_new.py","util_new.py")

node.upload_file("testprogs/hello.py","hello.py")
node.upload_file("testprogs/GBN_Client_buffer.py","GBN_Client_buffer.py")
node.upload_file("testprogs/GBN_Server_buffer.py","GBN_Server_buffer.py")

#node.upload_file("testprogs/test_file_10KB.txt","test_file_10KB.txt")
#node.upload_file("testprogs/test_file_100KB.txt","test_file_100KB.txt")
#node.upload_file("testprogs/test_file_1MB.txt","test_file_1MB.txt")
```

2. Run Experiment

b) Modified the “GBN_Client.py” into “GBN_Client_buffer.py”:

- Modified the code related to “window moving”

```
if ack == 0:
    print("Filename packet acknowledged by server.")

# advance window (if possible)
while len(window) > 0:
    first_seq, first_pkt = window[0]
    #h = PacketHeader(pkt[:PacketHeader.header_len])
    if first_seq <= ack:
        window.pop(0)
        del buffered_packets[first_seq]
        print(f"{first_seq} ACK received, removing from window ")
    else:
        break
```

- Modified the code related to “timeout, retransmit only earliest unacknowledged packet instead of all packets in the window”

```
        print(f"Find packet with seq <= acknowledged. Name is {query}. Format {seq}")
        break

    except socket.timeout as e:
        ## Timeout occured ##
        if len(window) > 0:
            earliest_seq = min(buffered_packets.keys()) # Find the earliest unacknowledged
packet
            client_socket.sendto(buffered_packets[earliest_seq], (args.address, args.port))
            print(f"Timeout:Retransmitting only the earliest unackowledged packet
{earliest_seq}")
        else:
            print(f"TIMEOUT: No unacknowledged packets to resend.")
            client_socket.settimeout(timeout)

# Close the client socket and file
```

there are also some other minor change, for details, please review the “GBN_Client_buffer.py”

c) Modified the “GBN_Server.py” into “GBN_Server.py”:

- Modified the code related to “check whether the new received packet has been buffered”, “receiver out-of-order packet and store in buffer” and “send back ACK”

```

86         "quitting".format(pkt_header.seq_num, final_timeout))
87     outfile.close()
88     end_time = time.time()
89     server_socket.settimeout(final_timeout)
90
91     expected_seq += 1
92
93     # new add codes to check buffered packets
94     while expected_seq in buffered_packets:
95         buffered_pkt = buffered_packets.pop(expected_seq)
96         outfile.write(buffered_pkt)
97         bytes_rcvd += len(buffered_pkt)
98         print(f"Expected sequence {expected_seq} has already been buffered.")
99         expected_seq += 1
100
101     elif pkt_header.seq_num > expected_seq:
102         print(f"[BUFFER] Out-of-order packet {pkt_header.seq_num} received, buffering it")
103         buffered_packets[pkt_header.seq_num] = data
104         print(f"[BUFFER STATE] Current buffered packets: {sorted(buffered_packets.keys())}")
105
106     # Send ACK
107     print(f"[ACK] Sending ACK {expected_seq-1}")
108     ack_header = PacketHeader(type=PacketHeader.TYPE_ACK, seq_num=(expected_seq-1), length=0)
109     server_socket.sendto(bytes(ack_header), addr)

```

there are also some other minor change, for details, please review the “GBN_Server_buffer.py”

Here is the screenshot of the Client node, from this screenshot, we can see:

It sends out the 1st filename packet;

From packets 0 to 15, receiver gave ACKs feedback successfully, so once sender got these ACKs for 0 -15, it just removing it from its window;

For packet 16, sender got more than 3 times ACKs(ACK 15), which means receiver did not got packet 16, so with timeout, sender retransmitted only this earliest unacknowledged packet 16 instead of other packets which sequence number are bigger than 16 but have not got 3 duplicated ACKs(like packet 19, 20 etc.)

After retransmitted packet 16 and got ACKs of packet 16, sender just remove it from the window and keep moving forward.

```
CreateSlice.ipynk × RDTLab.ipynb × ubuntu@client: ~ × ubuntu@server: - × GBF

ubuntu@client:~$
ubuntu@client:~$ python3 GBN_Client_buffer.py -a 10.131.2.2 -f test_file_1MB.txt
Send:Filename_PacketHeader: type=4, seq_num=0, length=17, filename=test_file_1MB.txt
Send:PacketHeader: type=1, seq_num=1, length=1000
Send:PacketHeader: type=1, seq_num=2, length=1000
Send:PacketHeader: type=1, seq_num=3, length=1000
Send:PacketHeader: type=1, seq_num=4, length=1000
Send:PacketHeader: type=1, seq_num=5, length=1000
Received ACK: seq=0
Filename packet acknowledged by server.
Received ACK: seq=1
1 ACK received,removing from window
Send:PacketHeader: type=1, seq_num=6, length=1000
Received ACK: seq=2
2 ACK received,removing from window
Send:PacketHeader: type=1, seq_num=7, length=1000
Received ACK: seq=3
3 ACK received,removing from window
Send:PacketHeader: type=1, seq_num=8, length=1000
Received ACK: seq=5
4 ACK received,removing from window
5 ACK received,removing from window
Send:PacketHeader: type=1, seq_num=9, length=1000
Send:PacketHeader: type=1, seq_num=10, length=1000
Received ACK: seq=6
6 ACK received,removing from window
Send:PacketHeader: type=1, seq_num=11, length=1000
Received ACK: seq=7
```

```
Received ACK: seq=14
14 ACK received, removing from window
Send:PacketHeader: type=1, seq_num=19, length=1000
Received ACK: seq=15
15 ACK received, removing from window
Send:PacketHeader: type=1, seq_num=20, length=1000
Received ACK: seq=15
Received ACK: seq=15
Received ACK: seq=15
Timeout:Retransmitting only the earliest unacknowledged packet 16
Received ACK: seq=20
16 ACK received, removing from window
17 ACK received, removing from window
18 ACK received, removing from window
19 ACK received, removing from window
20 ACK received, removing from window
Send:PacketHeader: type=1, seq_num=21, length=1000
Send:PacketHeader: type=1, seq_num=22, length=1000
Send:PacketHeader: type=1, seq_num=23, length=1000
Send:PacketHeader: type=1, seq_num=24, length=1000
Send:PacketHeader: type=1, seq_num=25, length=1000
Received ACK: seq=21
21 ACK received, removing from window
Send:PacketHeader: type=1, seq_num=26, length=1000
```

Here is the screenshot of the Server node, from this screenshot, we can see:

It received the 1st filename packet successfully;

From packet 0 to 15, receiver receive them successfully and sent ACKs back normally;

Suddenly, due to network status, it received packet 17,18,19,20 out of order, which maybe since the packet 16 is lost in the network transmission, instead of throwing away these out of order packets, receiver stored them in its buffer, until receiver got packet 16 again, receiver just use the buffered packet 17-20 and keep moving its window forward.

```
ubuntu@server:~$  
ubuntu@server:~$  
ubuntu@server:~$ python3 GBN_Server_buffer.py  
Opened Server socket  
Bound socket on port 12000  
[RECEIVED] PacketHeader: type=4, seq_num=0, length=17, filename=test_file_1MB.txt  
[ACK] Sent ACK for filename packet: seq=0  
[RECEIVED] expected_seq=1, receive packet 1,  
[ACK] Sending ACK 1  
[RECEIVED] expected_seq=2, receive packet 2,  
[ACK] Sending ACK 2  
[RECEIVED] expected_seq=3, receive packet 3,  
[ACK] Sending ACK 3  
[RECEIVED] expected_seq=4, receive packet 4,  
[ACK] Sending ACK 4  
[RECEIVED] expected_seq=5, receive packet 5,  
[ACK] Sending ACK 5  
[RECEIVED] expected_seq=6, receive packet 6,
```

.....

```
[RECEIVED] expected_seq=14, receive packet 14,  
[ACK] Sending ACK 14  
[RECEIVED] expected_seq=15, receive packet 15,  
[ACK] Sending ACK 15  
[BUFFER] Out-of-order packet 17 received, buffering it  
[BUFFER STATE] Current buffered packets: [17]  
[ACK] Sending ACK 15  
[BUFFER] Out-of-order packet 18 received, buffering it  
[BUFFER STATE] Current buffered packets: [17, 18]  
[ACK] Sending ACK 15  
[BUFFER] Out-of-order packet 19 received, buffering it  
[BUFFER STATE] Current buffered packets: [17, 18, 19]  
[ACK] Sending ACK 15  
[BUFFER] Out-of-order packet 20 received, buffering it  
[BUFFER STATE] Current buffered packets: [17, 18, 19, 20]  
[ACK] Sending ACK 15  
[RECEIVED] expected_seq=16, receive packet 16,  
Expected sequence 17 has already been buffered.  
Expected sequence 18 has already been buffered.  
Expected sequence 19 has already been buffered.  
Expected sequence 20 has already been buffered.  
[ACK] Sending ACK 20  
[RECEIVED] expected_seq=21, receive packet 21,  
[ACK] Sending ACK 21  
[RECEIVED] expected_seq=22, receive packet 22,  
[ACK] Sending ACK 22  
[RECEIVED] expected_seq=23, receive packet 23,  
[ACK] Sending ACK 23
```